

INITIATION AU LOGICIEL SAS cours n°3

MASTER MQME

Nicolas Cabaj

SOMMAIRE

1. Présentation du logiciel
2. Importation des données
3. L'étape Data et les proc print et contents
4. Les opérateurs arithmétiques et les fonctions SAS
5. Les fonctions conditionnelles et logiques
6. La proc sort
7. Les fusions de tables
8. La proc Format
9. La proc freq
10. La proc means
11. La proc univariate
12. Les Dates

PROC SORT (1/2)

But de la procédure:

Trier des tables selon une ou plusieurs variables dans l'ordre croissant ou décroissant.

Syntaxe:

```
Proc Sort data=table1 out=table2;  
By variable1;  
Run;
```

Ici, on crée la table *table2* qui correspond à la table *table1* triée dans l'ordre croissant de la variable *variable1*.

Par défaut, la proc sort trie les données dans l'ordre croissant.

Pour trier dans l'ordre décroissant il faut rajouter l'option *descending*

Syntaxe:

```
Proc Sort data=table1 out=table2;  
By descending variable1;  
Run;
```

Il n'est pas nécessaire de mettre l'option *out=table2*, dans ce cas, la table *table1* est écrasée par la nouvelle table triée.

L'option *nodupkey* permet de supprimer les doublons dans une table (la première observation est conservée et les autres sont supprimées).

Syntaxe:

```
Proc Sort nodupkey data=table1 out=table2;  
By descending variable1;  
Run;
```

Pour éviter de détruire les données initiales, il est préférable de toujours mettre un *out=* lorsque l'on utilise le *nodupkey*

PROC SORT (2/2)

Il est également possible de trier selon plusieurs variables les données

Syntaxe:

```
Proc Sort data=table1 out=table2;  
By variable1 variable2;  
Run;
```

Ceci est très utile lorsque la variable *variable1* prend plusieurs fois la même valeur, ainsi les données sont ensuite triées selon *variable2*.

Utilités de la Proc Sort:

- Créer de nouvelles tables triées (avec ou sans doublons) selon une ou plusieurs variables
- Etape préliminaire à la fusion de plusieurs tables SAS
- Etape préliminaire à la création d'agrégats dans les procédures statistiques (*by*)
- Etape préliminaire à l'utilisation des mots clés *first* et *last* dans les étapes data

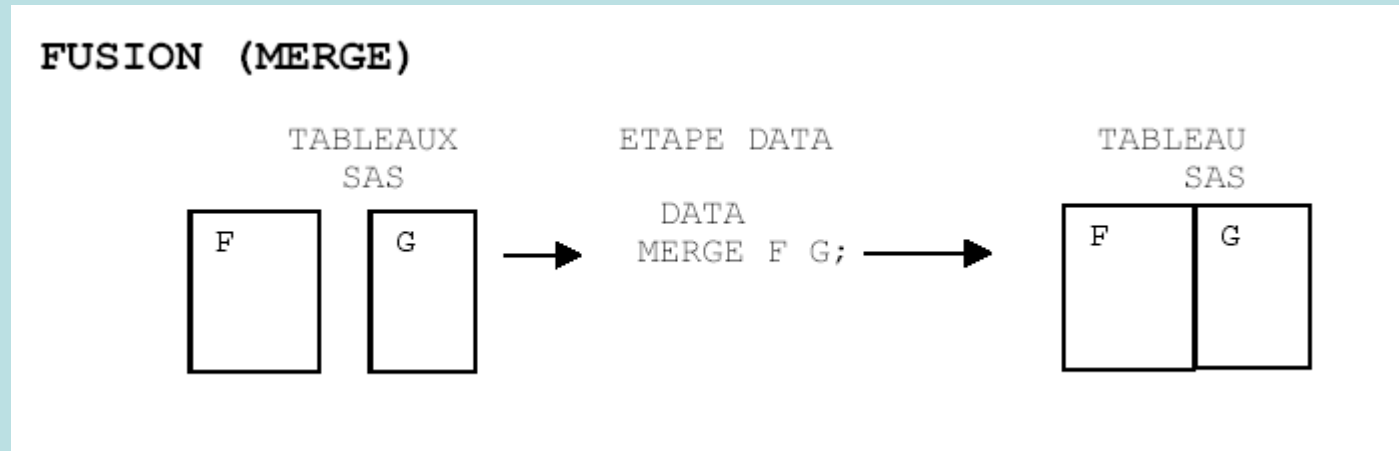
Exemple : utilisation de *first* dans une étape Data.

Le mot clé *First* permet d'identifier la première observation d'une variable. Il nécessite que la table soit triée selon cette variable. L'instruction *by variable;* doit être aussi présente dans l'étape data.

```
Proc sort data commandes;  
by num_client date_commande; run;  
  
Data commande1 autres_commandes;  
Set commandes;  
By num_client;  
if First.num_client then output commande1;  
else output autres_commandes;  
run;
```

Les fusions de tables (1/2)

Il est parfois nécessaire de regrouper des données présentes dans deux tables différentes et de les mettre dans une nouvelle table. On parle de fusion de tables.



L'instruction *Merge* permet cette fusion.

Syntaxe:

```
DATA nouvelle_table;  
MERGE table1 (IN=a)  
table2 (IN=b) ...  
tableN (IN=n) ;  
BY var_cle ;  
if a and b and ... and n ;  
RUN;
```

On fusionne les tables *table1 table2... tableN* sur la clé *var_cle* en conservant les lignes présentes dans chacune des tables.

L'option *(in=a)* permet de donner un nom raccourci à la table1 → l'instruction *if a and b* veut donc dire: si les mêmes données de la variable *var_cle* sont dans la table *table1* et *table2*.

Les fusions de tables (2/2)

Il est primordial d'effectuer un proc sort de toutes les tables que l'on fusionne selon la variable de clé de fusion avant de lancer le Merge.

Il est possible de rajouter les mêmes instructions dans un *Merge* que dans une étape Data classique.

```
DATA nouvelle_table;  
MERGE table1 (IN=a)  
table2 (IN=b) ;  
BY var_cle ;  
if a and b ;  
variable3 = max(variable1, variable2)  
if variable3>0;  
drop variable3;  
RUN;
```

Les différents types de fusions:

- Jointure totale : *if a and b*; on conserve les lignes présentes dans les 2 tables
- Jointure partielle : *if a*; on ne conserve que les lignes présentes dans la table a (quelles soient ou non dans la table b)
- Jointure complète : *if a or b*; on conserve tous les enregistrements de a et b qu'ils soient communs ou pas.

PROC FORMAT (1/3)

Préalable:

Les formats vont permettre d'indiquer à SAS la manière de lire ou d'écrire des données. Certains formats sont prédéfinis dans SAS. D'autres formats sont créés par l'utilisateur. Les formats ont des formes différentes selon qu'ils sont destinés à des variables numériques ou à des variables caractères.

Les formats prédéfinis dans SAS

1 Les formats numériques : Le format w.d

Il est sous la forme W.D. où :

- W (width) désigne la longueur totale en nombre de colonnes du champ contenant la valeur
- D (décimal) désigne le nombre de décimales éventuelles

On va associer à une variable numérique un format qui précisera la manière de l'écrire : on pourra préciser l'espace que doit prendre la variable, et le nombre de décimales que l'on souhaite afficher.

Exemple : **format salaire 7.2;** salaire désigne le nom de la variable et 7.2 est le format associé.

Le format 7.2 attribué à la variable "salaire" signifie que les valeurs du salaire seront affichées sur 7 colonnes, dont les deux dernières seront réservées aux décimales. La valeur entière du salaire s'affichera sur 4 colonnes, la 5ème colonne contiendra le point décimal et les deux dernières contiendront les décimales. Ainsi si la valeur du salaire était de 1234,5678 , lorsque le format 7.2 sera appliqué la valeur affichée sera 1234,56

PROC FORMAT (2/3)

2_ Les formats caractère : Le format \$w.

Il est sous la forme \$w. où W (width) désigne la longueur totale en nombre de colonnes du champ contenant la valeur.

Le signe \$ (dollar) est **indispensable**, sinon il s'agit d'un format numérique.

La chaîne de caractères est justifiée à gauche, sans espace ou blanc devant.

On va associer à une variable caractère un format qui précisera la manière de lire la variable ou de l'écrire : on pourra indiquer à SAS de ne lire que les 4 premières lettres des valeurs d'une variable, ou encore d'afficher des prénoms sur 12 colonnes (au lieu de 8 par défaut sous SAS) afin qu'ils ne soient pas tronqués par exemple.

Exemple

`format jeu $4.;` : "jeu" désigne le nom de la variable et \$4. est le format associé

Il signifie que l'on désire réserver 4 colonnes pour imprimer la valeur de la variable jeu.

Ainsi, si la valeur de la variable jeu est *trucmuche*, dans le tableau SAS la valeur affichée sera :truc

Les formats créés par l'utilisateur

Présentation : la procédure FORMAT

SAS permet de créer des formats personnalisés et de les conserver dans une bibliothèque où l'on pourra venir puiser selon les nécessités du travail.

La procédure FORMAT permet de créer des formats de lecture ou d'écriture de données qui pourront ensuite être associés aux variables.

La proc Format est surtout utilisée dans les proc Freq et Means pour améliorer l'affichage des résultats.

PROC FORMAT (3/3)

La procédure **FORMAT** a deux utilités :

- *Attribuer des libellés aux modalités des variables :*

Dans les tableaux, les variables sont codées : par exemple le sexe est codé 1, 2 ou H, F. Les formats vont permettre de faire correspondre des libellés aux modalités d'une variable. Ainsi pour la variable "sexe" le format fera correspondre le libellé 'homme' au code 1 et le libellé 'femme' au code 2.... ce qui rendra les tableaux plus clairs !

- *Effectuer des regroupements :*

Les formats vont aussi permettre d'effectuer des regroupements. Par exemple, on connaît l'âge de tous les individus et on veut effectuer des regroupements en créant des classes d'âges de 5 ans.

Syntaxe de programmation:

```
PROC FORMAT;  
VALUE nom du format  
    val1 = "Groupe1"  
    val2, val3, val4 = "Groupe2"  
    val5-val9 = "Groupe3" ...  
    ;  
RUN;
```

Si le format porte sur des variables caractères alors il faut précéder de \$ le nom du format et les valeurs val1 val2... sont entre ""

Exemple :

```
proc format;  
value $sexelib  
    " 1 " = "Homme"  
    " 2 " = "Femme"  
    ;  
run;
```

Une proc Format peut contenir plusieurs instructions value.

Il existe 3 valeurs particulières :

- **LOW** : désigne la valeur la plus petite (hormis la valeur manquante)
- **HIGH** : désigne la valeur la plus grande
- **OTHER** : désigne les valeurs non précédemment décrites.

PROC FREQ (1/5)

Préalable :

Les procédures *FREQ* servent à réaliser des dénombrements (= tris à plat) et des croisements (=tris croisés) de variables.

Les procédures *Freq* ne s'utilisent que sur des variables qualitatives ou des variables quantitatives prenant peu de valeurs (age de enfants).

Syntaxe

```
proc freq data=nom_table;  
tables liste_des_variables ;  
run ;
```

Exemple :

```
proc freq data=donnees ;  
tables age_enfant;  
run ;
```

Exemples de dénombrements

- D'une variable : *tables var ;*
- De plusieurs variables : *tables var1 var2 var3 ;*
- D'une liste de variables : *tables var1 - - var7 ;*

Exemples de tri croisés

- De deux variables *tables var1 * var2 ;* (var1 en ligne ; var2 en colonne)
- De trois variables *tables var1*var2*var3 ;* var1 est en page, var2 en ligne et var3 en colonne
- D'une variable avec plusieurs variables *tables var1 * (var2 var3) ;* Tri croisé successif de var1 avec var2 puis var1 avec var3
- Pour un dénombrement brut on obtient pour chaque modalité de la variable les fréquences, les pourcentages, les fréquences cumulées et les pourcentages cumulés.
- Pour un croisement on obtient un tableau avec les fréquences, les pourcentages par rapport au total, les pourcentages en ligne, les pourcentages en colonne.

PROC FREQ (2/5)

Les options:

Il existe de nombreuses options dans SAS qui permettent d'accroître les possibilités de la procédure. Ces options peuvent s'écrire :

- **sur la ligne PROC**

- *order = freq* modalités du tableau ordonnées en fonction de leur effectif

Exemple :

```
proc freq data=donnees order=freq ;  
tables sexe * csp ;  
run ;
```

- **sur la ligne de requête, introduites par une barre oblique /.**

On distingue les options d'affichage et les options de calcul:

- **Options d'affichage**

- *nocum* supprime l'affichage des effectifs et pourcentages cumulés
- *norow* supprime l'affichage des pourcentages en ligne
- *nocol* supprime l'affichage des pourcentages en colonne
- *nopercent* supprime l'affichage des pourcentages par rapport au total
- *nofreq* supprime l'affichage des effectifs
- *List* affiche le tableau sous forme d'une liste
- *noprint* supprime l'affichage des résultats
- *out = tab* crée une table en sortie, avec les croisements en liste

Exemple :

```
proc freq data=donnees;  
tables sexe * csp / nocol norow;  
run ;
```

PROC FREQ (3/5)

- **Options de calcul**

- *chisq* demande de test du chi-deux. SAS indique la probabilité de se tromper en rejetant l'hypothèse d'indépendance des variables
- *cellchi2* contribution des cases au chi-deux
- *all* demande de toutes les statistiques

Exemple :

```
proc freq data=donnees;  
tables sexe * csp / norow chisq;  
run ;
```

Remarque : si on fait plusieurs tris avec les mêmes options on n'écrit qu'une seule instruction *tables*, sinon on peut inscrire des instructions *tables* successives avec des options différentes.

Exemple :

```
tables var1*var2 var3*var4 / nocol ;  
tables var1*var3/nocum norow ;  
tables var2*var4/chisq;
```

L'option *missing* prend en compte les valeurs manquantes dans le tableau

Exemple :

```
proc freq data=donnees;  
tables sexe * csp / missing;  
run ;
```

L'option *out* permet de sauvegarder les tris à plats ou croisés dans une nouvelle table

Exemple :

```
proc freq data=donnees;  
tables sexe / out=effectif_sexe;  
run ;
```

PROC FREQ (4/5)

Notions complémentaires

- **Pondération**

Lorsque le fichier de données contient une variable de pondération, l'instruction *weight* permet de l'appliquer.

Exemple :

```
proc freq data=donnees ;  
tables sexe * csp ;  
weight pond ;  
run ;
```

- **Titres**

Un titre peut être inséré dans chaque procédure, ce qui facilite la lecture du tableau de résultat. Il est introduit par le mot-clé *title* et est inscrit entre guillemets. Il se place généralement entre la ligne d'appel de la procédure et la ligne des requêtes.

Exemple :

```
proc freq data=donnees ;  
title 'Croisement du sexe et de la catégorie socio-professionnelle' ;  
tables sexe * csp ;  
run ;
```

Remarque : une instruction *title* reste valable jusqu'à la prochaine instruction *title*

PROC FREQ (5/5)

Notions complémentaires

- **Formats**

Comme vu dans le chapitre sur la proc Format, il est possible d'affecter un format aux variables que l'on étudie. Ce format va permettre d'améliorer l'affichage des modalités

Exemple :

```
proc format;  
value $sexelib  
" 1 " = "Homme"  
" 2 " = "Femme"  
;  
run;
```

```
proc freq data=donnees ;  
tables sexe * csp ;  
format sexe $sexelib. ;  
run
```

- **agrégats**

Il est possible d'effectuer des calculs agréger selon une autre variable avec l'instruction *by*. Cela permet souvent une lecture plus simple des données. Attention, il faut que la table soit triée selon cette variable avant l'exécution de la *proc freq*.

Exemple :

```
proc sort data=donnees; by sexe;  
run;  
proc freq data=donnees ; by sexe;  
tables csp ;  
run ;
```

PROC MEANS (1/7)

Préalable :

La procédure *means* permet de calculer la moyenne de variables numériques ainsi que d'autres paramètres de distribution.

Syntaxe générale

```
proc means data = donnees ;  
var age ;  
run ;
```

L'instruction `proc means` est suivie du nom de la base qui contient les données, introduite par un `data =`

Le mot clé *var* introduit la variable dont on souhaite calculer la moyenne. Si on souhaite

calculer la moyenne de plusieurs variables, il suffit de nommer ces variables après l'instruction `var` en les séparant par un espace.

run ; indique la fin de la séquence de programme.

Résultats : par défaut, SAS affiche les informations suivantes : nombre

The MEANS Procedure

Analysis Variable : age age

N	Mean	Std Dev	Minimum	Maximum
10	11.5000000	1.5811388	9.0000000	14.0000000

On peut obtenir une moyenne pondérée par une variable (la variable de pondération « poids »)

PROC MEANS (2/7)

Options de la proc means :

On peut introduire des options qui interdisent l'affichage des options par défaut et n'affichent que celles qui vont être spécifiées :

- le nombre d'observations ayant un contenu non manquant *n*
- le nombre d'observations ayant un contenu manquant *nmiss*
- les valeurs minimum et maximum *min max*
- l'intervalle *range*
- la somme des valeurs *sum*
- la moyenne *mean*
- la variance *var*
- l'écart-type *std* (comme standard deviation)
- le nombre x de décimales à afficher *maxdec=x*

Les options s'écrivent sur la ligne du *proc means*, après le nom du fichier et avant le *;* avec un espace comme séparateur.

Exemple :

```
proc means data = donnees mean maxdec=1 sum ;  
var age ;  
run ;
```

The MEANS Procedure

Analysis Variable : age age

Mean	Sum
11.5	115.0

PROC MEANS (3/7)

Calculer une moyenne pour des groupes de population:

Il est possible de calculer une moyenne non pas sur toute la population mais sur des sous-ensembles de cette population, identifiés par une variable « de classement ». La variable sexe permet par exemple de définir un groupe « hommes » et un groupe « femmes ».

La variable qui classe la population en sous-populations est introduite par le mot clé *class*.

Ainsi, pour calculer la moyenne d'âge des hommes et celle des femmes, la syntaxe sera :

```
proc means data=donnees ;  
class sexe ;  
var age ;  
run ;
```

SAS éditera alors plusieurs lignes de résultats : une ligne pour les hommes et une ligne pour les femmes.

Il peut être utile d'associer un format à cette variable de classement, format qui devra avoir été défini au préalable. Le format permettra une lecture plus claire des résultats.

The MEANS Procedure

Analysis Variable : age age

sexe	N Obs	N	Mean	Std Dev	Minimum	Maximum
Femme	4	4	11.8	1.0	11.0	13.0
Homme	6	6	11.3	2.0	9.0	14.0

PROC MEANS (4/7)

Calculer des moyennes selon des combinaisons de variables:

L'instruction *TYPES* permet de définir quelles moyennes doivent être calculées et selon quelles combinaisons de variables. Les combinaisons de variables s'écrivent grâce à une étoile *
L'instruction *TYPES* doit obligatoirement être précédée d'une instruction *CLASS*. Toutes les variables qui seront utilisées dans l'instruction *TYPES* doivent **obligatoirement au préalable** être déclarées dans l'instruction *CLASS*.

Exemple avec une combinaison du sexe (homme/femme) et de l'activité sportive pratiquée (natation/gym)

```
proc means data=libname.donnees ;  
format sexe $genre. ;  
class sexe activite ;  
Types ( ) sexe sexe * activite ;  
var age ;  
run ;
```

Cette programmation permet d'obtenir :

- la moyenne d'âge de l'ensemble de l'échantillon ()
- la moyenne d'âge pour les hommes et pour les femmes *sexe*
- la moyenne d'âge pour chacune des combinaisons des variables sexe et activite *sexe*activite*

PROC MEANS (5/7)

The MEANS Procedure

Analysis Variable : age age

N Obs	Mean
10	11.5

Analysis Variable : age age

sexe	N Obs	Mean
Femme	4	11.8
Homme	6	11.3

Analysis Variable : age age

sexe	activite	N Obs	Mean
Femme	gym	2	11.5
	natation	2	12.0
Homme	gym	2	12.0
	natation	4	11.0

PROC MEANS (6/7)

Créer un tableau de sortie :

Il peut être utile de sauvegarder dans une nouvelle base de données les calculs effectués. Ils seront alors « réutilisables ». Ceci est possible grâce à l'instruction *output out =* suivie du nom de la nouvelle base de données.

Dans la base « valeurs », SAS génère par défaut trois nouvelles variables :

STAT contient le nom des statistiques figurant dans le tableau

FREQ contient le nombre d'observations sur lequel les calculs ont été effectués

TYPE donne le nombre de critères de tri utilisés pour le calcul des statistiques. Par défaut elle vaut 0 ; si on utilise l'instruction *class* elle vaudra 1.

Pour faire apparaître uniquement la statistique souhaitée, on indiquera sur la ligne *output out =* , en fin de ligne mais avant le ; le nom de la statistique souhaitée suivie du signe =.

La statistique prend alors le nom de la variable sur laquelle elle porte.

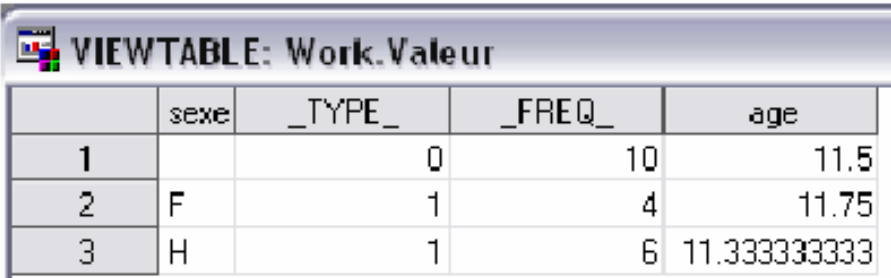
Exemple :

```
proc means data=donnees ;  
class sexe ;  
var age ;  
Output out = valeur mean = ;  
run ;
```

Dans cet exemple, la moyenne d'âge est calculée pour les hommes et pour les femmes.

SAS crée également un tableau nommé « valeur ».

L'indication *mean =* à la fin de la ligne permet de ne conserver que l'affichage de la moyenne.



	sexe	_TYPE_	_FREQ_	age
1		0	10	11.5
2	F	1	4	11.75
3	H	1	6	11.33333333

PROC MEANS (7/7)

Pour demander la statistique de plusieurs variables en même temps, il suffira d'indiquer leur nom entre parenthèses avant le signe `=`.

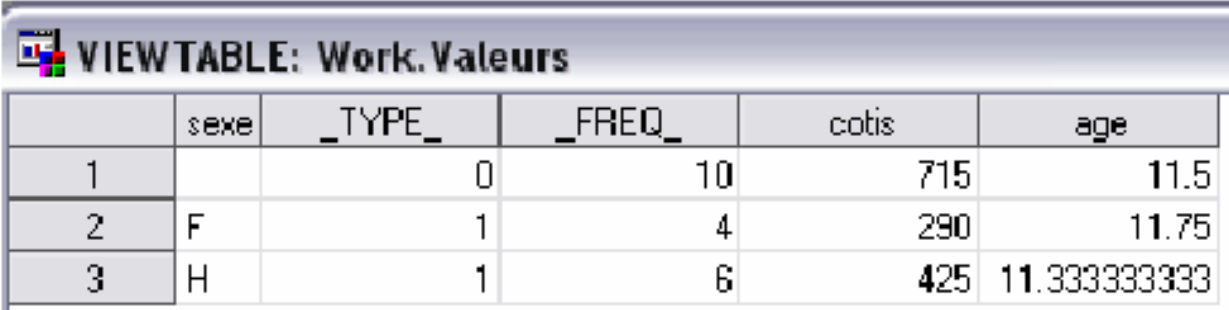
Exemple : *output out=valeurs mean (poids taille) = ;*

Il est possible de calculer des statistiques différentes sur des variables différentes dans une même base de données :

Exemple :

```
proc means data=donnees ;  
class sexe ;  
var cotis age ;  
Output out = valeurs sum (cotis) = mean (age) = ;  
run ;
```

Dans cet exemple, la base *valeurs* contiendra la somme des cotisations ainsi que la moyenne d'âge pour chaque sexe et pour l'ensemble.



	sexe	_TYPE_	_FREQ_	cotis	age
1		0	10	715	11.5
2	F	1	4	290	11.75
3	H	1	6	425	11.33333333

L'option *noprint* à la fin de la ligne *proc means* et avant le `;` permet de ne pas afficher les résultats dans l'output, la procédure sert alors uniquement à créer la base de données. Très utile lors de l'utilisation d'un class sur une variable ayant beaucoup de valeurs.

PROC UNIVARIATE (1/1)

Préalable :

La procédure *univariate* est très utilisée dans les études statistiques des variables quantitatives. Elle complète la proc *means* puisqu'elle fournit des tests statistiques sur les variables choisies.

Syntaxe générale

```
PROC UNIVARIATE Data=donnees;  
VAR variables;  
BY variables; (Permet la création de sous-groupes... le fichier devra avoir été trié avant)  
OUTPUT OUT=tab_sortie (fichier contenant autant d'individus que de modalités de BY et comme variables, les  
statistiques données par UNIVARIATE)  
RUN;
```

Autre intérêt de la proc *univariate* : il est possible d'obtenir les valeurs des déciles, centiles, quartiles, quintiles... des chaque variable quantitative.

Il est souvent très intéressant récupérer dans un fichier la valeur de chaque centile du fichier:

```
PROC UNIVARIATE Data=donnees;  
VAR variables;  
OUTPUT OUT=tab_sortie pctlpre=P_ pctlpts=0 to 100 by 1;  
RUN;
```

Utilisation des Dates (1/5)

Une date SAS est une valeur représentant le nombre de jours écoulés entre le 1er janvier 1960 (date de référence) et une date donnée. Grâce à cette présentation des dates, il sera possible de calculer des intervalles entre les dates. Les dates SAS peuvent être converties en jours, mois, années... Les dates SAS sont de type numérique.

Il est possible de transformer des dates écrites sous des formes diverses (01 JAN 82, 15/10/84, 251274 etc...) en dates SAS, exprimées en nombre de jours écoulés depuis le 1er janvier 1960. Il faut pour cela utiliser des « **informats** » dans une étape **DATA**.

Pour des dates représentées sous une forme alphanumérique, par exemple 1JAN1982, 01 JAN 82, 1 JAN 82, 1-JAN-82, l'informat à utiliser sera **DATEw.**, où **w** correspond à la taille nécessaire à l'affichage voulu. L'informat se déclare au moment de la constitution de la base de données, sur la ligne **INPUT**.

Dans l'exemple suivant, toutes les dates présentées sous une forme alphanumérique sont lues correctement (SAS reconnaît les séparateurs comme les espaces et les caractères spéciaux) grâce à l'informat [date10.] et sont toutes traduites dans la variable **date1** par la valeur 8036.

```
data base ;  
input date1 DATE10. ;  
cards ;  
1JAN1982  
01 JAN 82  
1 JAN 82  
1-JAN-82  
;  
run;
```

Utilisation des Dates (2/5)

DDMMYYw. *MMDDYYw.* *YYDDMMw.* sont des informats qui permettent d'écrire en date SAS des dates présentées sous forme numérique.

D représente le jour (« Day »)

M représente le mois (« Month »)

Y représente l'année (« Year »)

Dans cet exemple les dates sont présentées sous la forme jour mois année donc on utilise l'informat *DDMMYY8.* Ici toutes les dates sont traduites correctement par la valeur 8323.

```
data base ;  
input date1 DDMMYY8. ;  
cards ;  
151082  
15-10-82  
15/10/82  
;  
run ;
```

Lorsque dans une base de données on a des dates SAS, on peut leur appliquer un **format** pour identifier « en clair » à quelles dates elles correspondent. Si on n'utilise pas de format de date particulier lors de la réalisation de tableaux par exemple, c'est la date SAS (exprimée en nombre de jours écoulés depuis le 1er janvier 1960) qui figurera dans les tableaux.

Les formats à utiliser ont la même forme que les informats.

On utilisera un format de type *datew.* pour traduire une date SAS en alphanumérique, tandis qu'on utilisera un format de type *ddmmyyw.* (par exemple) pour traduire une date SAS en une date exprimée en numérique.

La syntaxe est la suivante : *FORMAT date1 ddmmyy8. ;*

Utilisation des Dates (3/5)

Exemples d'application de formats :

- *FORMAT date1 DATE5. => 15OCT*
- *FORMAT date1 DATE7. => 15OCT82*
- *FORMAT date1 DATE9. => 15OCT1982*
- *FORMAT date1 DDMMYY8. => 15/10/82*
- *FORMAT date1 MMDDYY8. => 10/15/82*

Fonctions sur les Dates:

TODAY() donne la date du jour.

date1 = TODAY() => date1 = 15763 si on est le 27 février 2003.

MDY(mois, jour, annee) transforme les informations d'une date stockée en trois variables en une nouvelle variable de type "date".

date1 = MDY(8, 18, 1961) => date1 = 595.

DAY(date), **MONTH** (date) et **YEAR**(date) transforme les informations de date stockée en une variable de type "date" en trois nouvelles variables.

date1 = TODAY() => date1 = 15763 si on est le 27 février 2003.

Jour = DAY(date1) => jour = 27

mois = MONTH(date1) => mois = 2

annee = YEAR (date1) => annee = 2003

Utilisation des Dates (4/5)

Fonction INTCK : calculer des intervalles

Grâce aux dates SAS, il est possible de calculer des intervalles entre une date1 et une date2. Cet intervalle pourra être exprimé en nombre (entier) de jours, mois, années...

La syntaxe de la fonction INTCK est la suivante :

var = INTCK (« interval »,from,to) ;

“from” et “to” indiquent les variables des dates entre lesquelles on souhaite calculer une durée. “interval” spécifie l’unité de mesure de l’intervalle. La valeur de ce paramètre peut être : DAY, MONTH, YEAR, WEEK...

Exemple :

dureevie = INTCK (« year »,nais,mort) ;

Dans cet exemple, on calcule dans la variable *dureevie* le nombre d’années écoulées entre le moment de la naissance (exprimé dans la variable *nais*) et le moment de la mort (exprimé dans la variable *mort*).

Remarque : le résultat de cette syntaxe donnera le nombre d’année arrondi à l’année supérieure. Pour être exact, il est préférable de calculer la durée en nombre de mois puis de diviser par 12 pour obtenir un nombre d’années, puis d’arrondir à l’entier inférieur grâce à la fonction FLOOR. Cela se traduit par la syntaxe suivante :

dureevie =FLOOR(INTCK (« month »,nais,mort)/12) ;

Utilisation des Dates (5/5)

Fonctions pour les heures:

TIME() donne l'heure du jour.

heure1 = TIME() => heure1 = 46024 si il est 12 h 47' 4"

DATETIME () donne la date et l'heure du jour

Var = datetime ()

HMS(heure, minute, seconde) transforme les informations d'une heure stockée en trois variables en une nouvelle variable de type "heure".

heure1 = HMS(12, 18, 10) => heure1 = 44290

HOURL(heure), **MINUTE** (heure) et **SECOND**(heure) transforme les informations d'heure stockée en une variable de type "heure" en trois nouvelles variables.

heure1 = TIME() => heure1 = 46040 si il est 12 h 47' 20"

heure = HOURL(heure1) => heure = 12

jour = MINUTE(heure1) => minute = 47

seconde = SECOND(heure1) => seconde = 20.07

Pour lire une « time SAS » il faudra lui appliquer le format *TIMEw.* tandis que pour lire une variable « datetime SAS », il faudra lui appliquer le format *DATETIMEw.*